Projektuppgift

Renoveringskalkylator

Programmering i C# .NET
DT071G

MITTUNIVERSITETET
Avdelningen för informationssystem och -teknologi

Författare: Alva Stern, alst2511@student.miun.se
Utbildningsprogram: Webbutveckling, 120 HP
Huvudområde: Datateknik
Termin, år: HT, 2026



Campus Härnösand Universitetsbacken 1, SE-871 88. **Campus Sundsvall** Holmgatan 10, SE-851 70 Sundsvall.
Campus Östersund Kunskapens väg 8, SE-831 25 Östersund.
Phone: +46 (0)771 97 50 00. Fax: +46 (0)771 97 50 01.

Innehållsförteckning

Innehållsförteckning 1
Introduktion 2
Problemdefinition 2
Teori 3
Metod 5
Brainstorming 5
Wireframes 5
Flödesschema 6
Objektorienterad programmering 7
SOLID - Single Responsibility Principle 7
Funktions- och valideringstestning 7
Konstruktion 8
Skapande av idé och lösning 8
UX / UI design 8
Flödesschema 9
ASP.NET och Razor Pages 9
Klasser och beräkningslogik 10
Formulärhantering och validering 11
Presentation av resultat 13
Styling och responsivitet 13
Resultat 14
Slutsatser 15

Introduktion

Denna projektuppgift har gått ut på att jag som student ska skapa en valfri applikation i C# tillsammans med ramverket .NET. Detta med syfte att jag ska få använda de tekniker och teorier som jag har lärt mig i denna kurs, och därmed få möjligheten att visa vad jag kan.

Applikationen som har tagits fram i detta projekt är en webbapplikation som kan räkna ut hur mycket material i form av färg, golv och takpanel som behövs till renovering. Användaren skriver in uppgifter i form av exempelvis yta, m² / paket och spill i procent, som programmet sedan använder för att räkna ut hur mycket material som behövs och hur många paket användaren behöver köpa.

Webbapplikationen har skapats med ASP.NET och Razor Pages, där .NET är plattformen som kör applikationen, ASP.NET är ramverket inom .NET som bygger webbapplikationen och Razor Pages som kopplar ihop HTML-koden med C#-koden. Dessa metoder användes för att jag skulle få möjligheten att utveckla det som jag lärt mig i kursen genom att skapa en webbapplikation och inte endast en konsolapplikation.

Problemdefinition

En renovering kan vara mycket komplex och det kan vara svårt att veta hur mycket material som behövs [1]. Eftersom jag dessutom själv är i slutet av en totalrenovering är jag medveten om dessa svårigheter. Därmed valde jag att lösa detta problem genom att skapa en webbapplikation som kan räkna ut hur mycket material som behövs vid en renovering. Målgruppen för denna projektuppgift blir därmed personer som renoverar och tycker det är svårt att veta hur mycket material som behöver köpas.

Teori

I denna del av rapporten beskrivs de begrepp, tekniker och ramverk som är bra att känna till för att kunna ta till sig innehållet på bästa sätt.

C#: Ett objektorienterat programmeringsspråk som är utvecklat av Microsoft och en del av .NET-plattformen [2]. C# kan användas till allt från att bygga webbtjänster och komplexa webbapplikationer, till spelutveckling och skrivbordsprogram [2].

.NET: En plattform för mjukvaruutveckling som används för att skapa och köra olika typer av program som exempelvis webbapplikationer och mobila appar [3]. Systemet består av olika komponenter som möjliggör att flera programmeringsspråk kan användas, exempelvis C# [3].

.NET framework: Ett ramverk och en programvaruplattform som är utformad för att underlätta programvaruutveckling [3]. Ramverket .NET innehåller olika verktyg, bibliotek och tjänster som gör det enkelt för utvecklare att skapa programvaror för Microsoft [3].

Wireframe: En wireframe är en skiss av en digital produkt (mobilapp, webbapplikation, etc.) som visar den grundläggande layouten, navigation och strukturen [4]. Den visar vart i gränssnittet olika komponenter finns, samt vart information ska visas [4].

High-fidelity: Om en wireframe eller prototyp görs med high-fidelity innebär detta att den görs mycket detaljerad [5]. Syftet med detta är att få användarupplevelsen och den slutgiltiga produkten att simuleras så nära som möjligt det slutgiltiga resultatet [5].

Flödesschema: Ett flödesschema är en grafisk beskrivning av en process som består av olika former för att beskriva ordningen i processen [6]. De används till att analysera, designa, dokumentera eller hantera en process eller ett program inom olika områden [6].

ASP.NET: Ett gratis ramverk för webben som utvecklare kan använda till att skapa webbplatser med HTML, CSS och JavaScript [7]. Med detta ramverk kan utvecklare dessutom skapa API:er, samt använda olika typer av realtidstekniker som exempelvis Web Sockets [7].

Razor Pages: Razor pages är en modell för programmering inom ASP.NET som gör det enkelt att bygga dynamiska webbplatser och applikationer som kräver mycket formulär [8]. Modellen har en sidbaserad struktur och anpassar sin kod och logik efter det [8].

Objektorienterad programmering: Objektorienterad programmering är en metod inom programmering som bygger på att koden organiseras i klasser och objekt och inte enbart i vanliga funktioner [9]. Detta gör att delar av koden kan återanvändas och ärvas av andra delar av koden [9].

Klasser: En klass används inom objektorienterad programmering och agerar som en mall för vilka egenskaper (exempelvis variabler) och beteenden (exempelvis metod eller funktion) ett objekt kan ha [9].

Objekt: Ett objekt är en specifik insats som har skapats utifrån en klass [9]. Exempelvis kan det finnas en klass som är *bok,* medan objektet som är en insats i klassen kan vara *Harry Potter och De Vises Sten* [9].

SOLID: SOLID är en uppsättning av principer för objektorienterad programmering [10]. Målet med principerna är att systemet ska bli enkelt att utveckla, förstå, underhållas och utökas [10].

Metod

I denna del av rapporten beskrivs vilka metoder som har använts för att komma fram till projektresultatet.

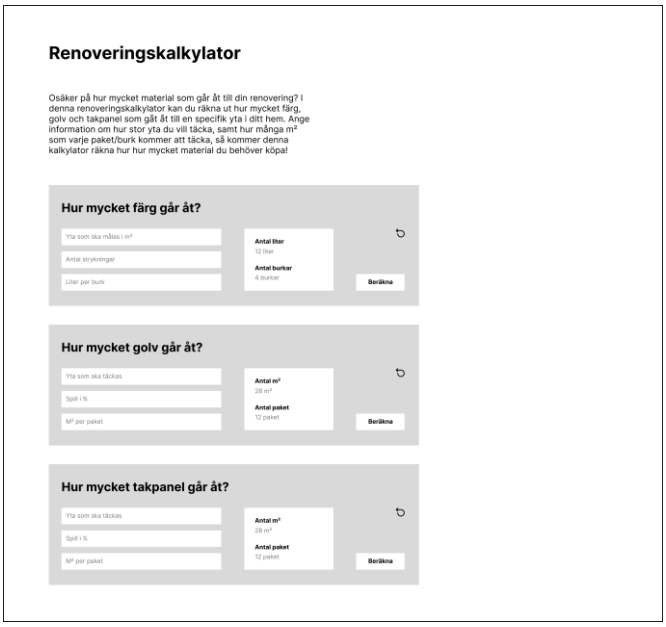
Brainstorming

Den första metoden som användes i detta projekt är brainstorming. Brainstorming är en kreativ metod som syftar till att komma på nya idéer och lösningar till problem [11]. Detta genomfördes med syfte att jag skulle komma upp med en idé som skulle vara lämplig för denna projektuppgift, samt skulle vara en lösning på ett problem.

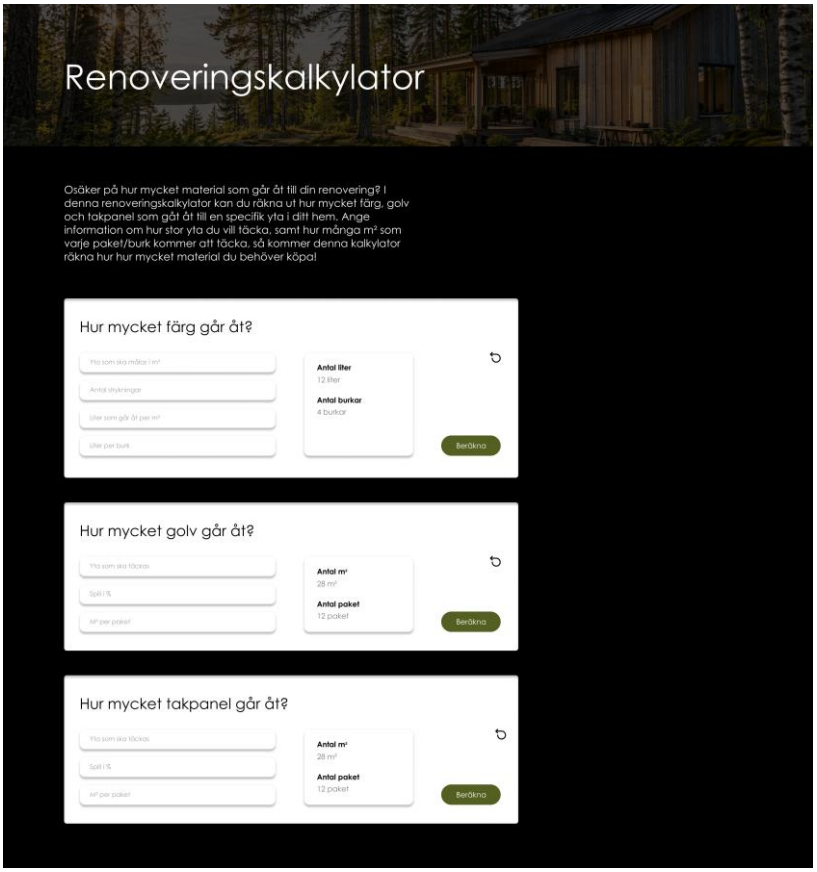
Brainstormingen genomfördes genom att jag tänkte mig in i situationer som är utmanande i min vardag. Jag funderade sedan på om dessa utmaningar kan lösas genom någon digital lösning inom ramarna för denna projektuppgift. Resultatet av brainstormingen blev att jag ska skapa en renoveringskalkylator som kan räkna ut hur mycket färg, golv och takpanel som går åt till en specifik yta.

Wireframes

Wireframes användes som metod med syfte att skapa en enkelt och användarvänlig informationsarkitektur, samt säkerställa att all information som behöver vara med i webbapplikationen kommer med. Genom detta fick jag en känsla över webbapplikationens layout, vilket gjorde att jag kunde få en översikt över gränssnittet och se till att webbapplikationens syfte kommer fram, samt att den inte bryter mot några konventioner.



Det skapades även en ytterligare wireframe med fokus på även det visuella och inte bara på strukturen och informationsarkitekturen. Detta genomfördes istället för en prototyp eftersom denna webbapplikation endast består av en sida och enkla funktioner. Därmed fanns det inget behov av att göra en prototyp eftersom interaktionen framgick tydligt genom mina wireframes och flödesscheman.

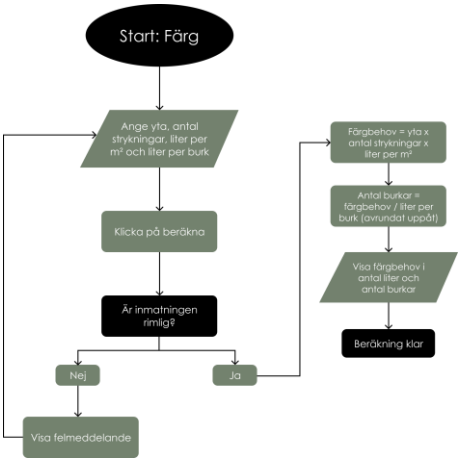


Flödesschema

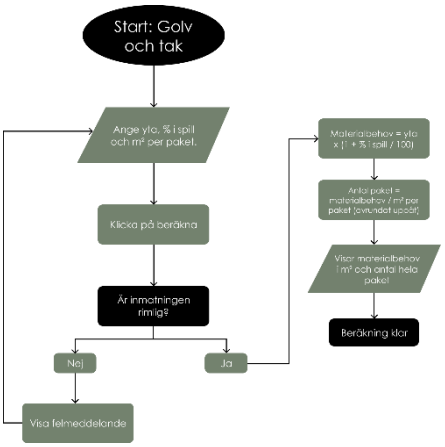
Flödesschema användes som metod eftersom den beskriver processen som användare går igenom i webbapplikationen för att genomföra syftet, i detta fall räkna ut hur mycket material som går åt i en renovering. Genom att skapa flödesscheman får jag en överblick över flödet i webbapplikationen och kan utifrån det göra en bedömning över komplexiteten och därmed hur enkel applikationen är att använda.

I detta projekt skapades två olika flödesscheman, en för uträkning av hur mycket färg som går åt och en för uträkning av golv och takpanel. Dessa skiljer sig åt eftersom de har olika typer av uträkningar. I dessa flödesscheman visualiseras alltså hur processen för funktionaliteten fungerar i webbapplikationen.

Flödesschema för färg



Flödesschema för golv / tak



Objektorienterad programmering

För att utveckla lösningen användes objektorienterad programmering som metod. Syftet med detta var att kunna organisera koden och göra beräkningarna återanvändbara. Två olika klasser skapades med ansvar för de olika beräkningarna. Beräkningarna separerades därmed från gränssnittet, vilket gjorde det möjligt att ändra på sidans styling utan att det påverkade beräkningslogiken. Objekt skapades sedan av klasserna, vilket gjorde att deras metoder kunde anropas med inmatade värden.

SOLID - Single Responsibility Principle

I projektet användes en av de fem designprinciperna inom SOLID som metod. Denna princip är Single Responsibility Principle (SRP) och användes med syfte att strukturera upp koden på ett lämpligt sätt. Målet var att varje klass endast skulle ha ett tydligt ansvarsområde för att koden skulle bli lättare att förstå, lättare att testa och ändra på utan att andra delar av webbapplikationen skulle påverkas.

Funktions- och valideringstestning

Under utvecklingens gång användes testning som metod för att säkerställa att allt fungerar som planerat. Applikationen testades manuellt av mig genom att mata in värden och jämföra applikationens svar med manuellt beräknade resultat för att se så beräkningarna fungerade som de skulle. Inmatningsvalideringarna testades genom att obligatoriska fält lämnades tomma och ogiltiga värden skrevs in. Detta gjorde det tydligt för mig om valideringarna fungerade som de skulle.

Konstruktion

I denna del av rapporten beskrivs upplägget av arbetet och den praktiska konstruktionen med webbapplikationen.

Skapande av idé och lösning

För att komma upp med en lösning till det identifierade problemet i denna projektuppgift, genomfördes som tidigare nämnt en session inom brainstorming. Det identifierade problemet är som sagt att personer som genomför en renovering kan tycka det är utmanande att veta hur mycket material som går åt till olika delar av renoveringen.

Genom att genomföra brainstorming och använda min kreativitet, framkom en lösning som gör det möjligt för användare att i en webbapplikation räkna ut hur mycket färg, golv och takpanel som går åt till en viss yta i hemmet som ska renoveras. Denna lösning blev lämplig eftersom det gjorde det möjligt för mig att skapa en applikation med C# som skulle vara lämplig för denna projektuppgift.

UX / UI design

För att komma upp med hur webbapplikationen skulle struktureras och se ut visuellt, implementerades UX/UI design som en del av lösningen. Här bestämdes vilken information som skulle finnas i webbapplikationen och hur den skulle visas. Detta gjorde jag genom att ta fram en wireframe där jag visade detta. Jag gjorde designval som exempelvis att webbapplikationen endast skulle ha en sida och därmed undvika navigation mellan sidorna, samt varje beräkning skulle ha en egen ruta.

Endast en sida implementerades eftersom webbapplikationen endast hade ett syfte som visas på startsidan. Därmed fanns det inget syfte att implementera flera sidor och en navigation som kan göra applikationen svårare att förstå. Sedan fick varje beräkning (färg, golv, takpanel) en egen ruta i gränssnittet med syfte att det skulle vara enkelt för användaren att skilja på uträkningarna och därmed göra applikationen tydlig och enkel att förstå för användarna.

För att komma upp med den visuella delen i webbapplikationen skapade jag en till wireframe so förutom att endast ha fokus på informationsarkitekturen, även skulle fokusera på det visuella. Jag satt och experimenterade med olika färger, typsnitt och bilder för att på så

sätt komma upp med den visuella aspekten till webbapplikationen. Här gjorde jag valet att ha en mörk bakgrund med ljusa rutor som beräkningarna ska göras i. Jag valde dessutom att implementera en grön färg på vissa knappar i gränssnittet för att få en jordnära och natur-känsla i webbapplikationen.

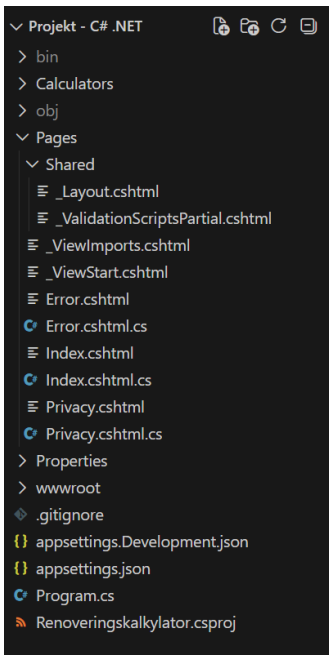
Flödesschema

För att säkerställa att processen för att genomföra en beräkning i webbapplikationen kommer fungera som planerat, skapades som tidigare nämnt två olika flödesscheman, en för varje beräkning. Detta gjordes genom att jag använde min wireframe och funderade över hur en användare behöver ta sig igenom webbapplikationen för att göra en beräkning av färg eller golv och takpanel.

Jag började med att fundera över vart användaren startat i processen och symboliserade detta i flödesschemat med en oval. Sedan använde jag vanliga fyrkantiga rutor för att visa vanliga olika steg i flödesschemat och parallellogram för att visa vart användaren ska mata in data, eller när användaren får data utmatad. Sedan kopplade jag ihop alla olika rutor med pilar som tydliggjorde ordningen för alla steg i flödesschemat.

ASP.NET och Razor Pages

För att utveckla min applikations-lösning till en webbapplikation använde jag mig av ASP.NET och Razor Pages. Jag skapade projektet genom att i terminalen för projektmappen skriva **dotnet new webbapp**. Genom detta fick jag grunden till projektets filstruktur och tilldelade filer där koden till webbapplikationen kunde implementeras.



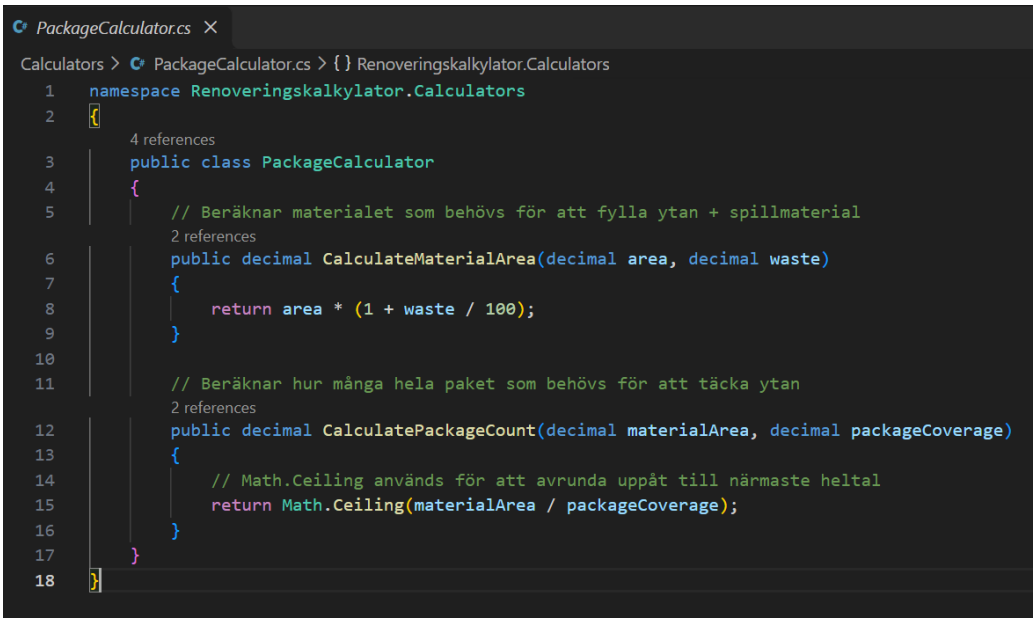
Sidans innehåll och de olika beräkningsformulären byggdes i filen **Index.cshtml**, medan hanteringen för formulären placerades i den tillhörande C#-filen **Index.cshtml.cs**. Varje formulär kopplades sedan i sin tur till en separat hanteringsmetod med attributet **asp-page-handler**. Ett exempel på detta är att golvformuläret kopplades till metoden **OnPostFloor** i filen **Index.cshtml.cs**.

HTML-strukturen för webbapplikationen placerades i filen **_Layout.cshtml**. Här började jag med att ta bort mallens navigation eftersom min webbapplikation endast innehåller en sida och därmed inte är i behov av någon navigation. Jag tog även bort mallens footer.

Klasser och beräkningslogik

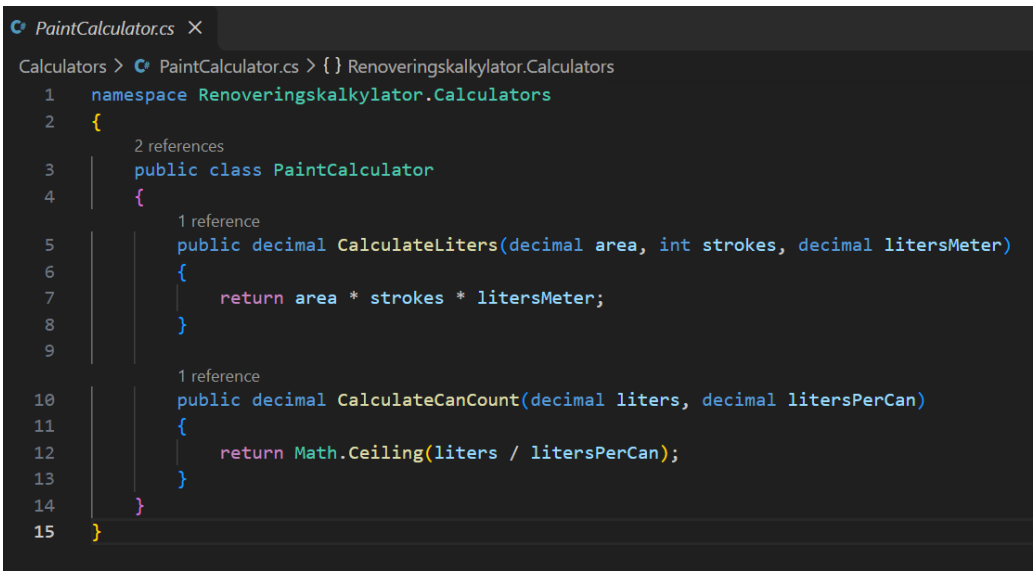
För att hantera beräkningslogistiken på bästa sätt delade jag upp dessa i två klasser, **PackageCalculator** för beräkning och golv och takpanel och **PainCalculator** för beräkning av färg. Dessa två klasser skapades i en mapp som jag döpte till **Calculators** och la i projektets root. Jag använde två olika klasser med varsitt tydligt ansvarsområde med syfte att hålla beräkningarna åtskilda och följa designprincipen SRP i SOLID.

Klassen **PackageCalculator** användes för både golv och takpanel eftersom materialet beräknades med samma algoritm. Metoden **CalculateMaterialArea** tar emot ytan och spillsprocent och returnerar materielbehovet utifrån formeln **area * (1 + waste / 100)**. Sedan tar metoden **CalculatePackageCount** och delar materielbehovet med antalet m² per paket för att komma fram till materielbehovet och antal paket som användaren behöver köpa till sin renovering. För att komma fram till de matematiska formlerna använde jag mig av generativ AI.



Klassen **PainCalculator** användes för att hantera färgberäkningarna. Metoden **CalculateLiters** tar emot yta, antal strykningar och hur mycket färg som går åt per m² i liter. Färgbehovet beräknas sedan genom först den matematiska formeln **area * strokes * litersMeter**. Sedan används

metoden **CalculateCanCount** för att ta reda på hur många burkar användaren behöver köpa genom formeln **liters / litersPerCan**.



I båda klasserna använde jag **Math.Ceiling** för att avrunda förpackningsantalet och antalet färgburkar uppåt till närmaste heltal. Detta eftersom material köps i hela paket, och om beräkningen kommer fram till att 7,5 paket golv behövs, räcker inte 7 paket, utan då behöver användaren köpa 8 paket för att materialet ska räcka.

För ytor, materialåtgång och beräkningsresultat använde jag mig av datatypen **decimal** för att tillåta decimaltal. För antalet strykningar använde jag istället datatypen **int** eftersom antalet strykningar måste anses som ett heltal.

De två klasserna används dessutom av sidans klass **IndexModel**, där objekt skapas och beräkningsmetoderna anropas. Här återanvänds klassen **PackageCalculator** eftersom golv och takpanel beräknas på samma i renoveringskalkylatorn.

Formulärhantering och validering

Hanteringen för formulären placerades i klassen **IndexModel** i filen **Index.cshtml.cs**. Varje beräkning fick ett eget formulär med metoden **post** som kopplades till en hanteringsmetod genom **asp-page-handler**. Namnet som anges i **asp-page-handlern** kopplas sedan till rätt metod i **Index.cshtml.cs**. Ett exempel på detta är att formuläret för golvberäkning namnges via **asp-page-handler** till **Floor**, vilket sedan är namnet som används i metoden i **Index.cshtml.cs**, **OnPostFloor**.

Metodnamnet blir **OnPostFloor** och inte bara **Floor** eftersom post används som metod i formulären.

```
<form method="post" asp-page-handler="Floor">
```

```
public void OnPostFloor(decimal floorArea, decimal floorWaste, decimal floorPackageCoverage)
```

För att kontrollera de värden som matas in i beräkningarna används både webbläsaren och C#-koden. I HTML används **required** i formulären för att visa på obligatoriska fält, **min** för det lägsta tillåtna värdet och **step** för att tillåta decimaler eller kräva heltal i input-fälten.

```
min="0" step="any" required/>
```

Fortsättningsvis kontrolleras de inskickade värdena i hanteringsmetoderna med **ModelState.IsValid**, vilket visar om de inskickade värdena kan tolkas som de förväntade datatyperna. Därefter kontrolleras värdena med villkor genom **if-satser** för att se att de inskickade värdena är tillåtna. Om ett värde inte är tillåtet läggs det till ett felmeddelande med **ModelState.AddModelError** och metoden avbryts.

```
// Kontrollerar att inmatningen kan tolkas som tal
if (!ModelState.IsValid)
{
    return;
}

// Kontrollera att värderna är tillåtna
if (floorArea <= 0 || floorWaste < 0 || floorPackageCoverage <= 0)
{
    ModelState.AddModelError(
        string.Empty,
        "Ange en yta och paketstorlek större än noll samt spill på minst 0 %.");

    return;
}
```

När värdena har passerat valideringarna skapas ett objekt och beräknings-klassernas metoder anropas. Resultatet sparas sedan i egenskaper i **IndexModel**.

```
// Skapar ett objekt av klassen PackageCalculator
PackageCalculator calculator = new PackageCalculator();

FloorMaterialArea = calculator.CalculateMaterialArea(floorArea, floorWaste);
FloorPackageCount = calculator.CalculatePackageCount(FloorMaterialArea.Value, floorPackageCoverage);
```

Presentation av resultat

Resultatet av beräkningarna visas i respektive uträknings resultatruta i webbapplikationen. För att läsa värdena från egenskaperna i **IndexModel** användes **Razor**. Detta gjordes genom uttryck som börjar med **@Model**. Resultatrutorna omges av en if-sats med ett villkor som kontrollerar om ett resultat finns med hjälp av **HasValue**. Därmed visas resultatet först efter att en beräkning har genomförts.

Vid visning av resultatet används **ToString(”0.##)**, vilket för att högst två decimaler visas. Antal paket eller burkar visas som hela förpackningar och avrundas uppåt. Även enheter skrivs ut tillsammans med värdena för att resultaten ska bli så tydliga som möjligt.

```
@if (Model.PaintLiters.HasValue)
{
    <article>
        <h3>Antal liter</h3>
        <p>@Model.PaintLiters.Value.ToString(format: "0.##") liter</p>

        <h3>Antal burkar</h3>
        <p>@Model.PaintCanCount burkar</p>
    </article>
}
```

Styling och responsivitet

För att ge styling åt min webbplats användes den givna CSS-filen **site.css**. Jag började med att ta bort den befintliga stylingen och ersätta den med min egen för att webbapplikationen skulle få den styling som var planerad. CSS-filen ligger i ramverkets mapp **wwwroot**. I denna mapp lade jag även till en mapp med images. I den mappen lade jag till bilden som jag använder i headern i min webbapplikation.

I CSS-filen använde jag mig av **@media** för att skapa responsivitet till min webbapplikation. Eftersom detta är en väldigt enkel webbapplikation med endast en sida valde jag att endast göra en brytning med en max-width på 768px.

Resultat

Resultatet av denna projektuppgift där en applikation med C# skulle skapas, blev en webbapplikation skapad med ASP.NET och Razor Pages. Webbapplikationen är en renoveringskalkylator som ska underlätta för personer som renoverar att ta reda på hur mycket material i form av golv, takpanel och färg som går åt i en renovering.

Problemet som definierades i detta projekt är att det kan vara svårt för personer som renoverar att veta hur mycket material som går åt. Med min framtagna webbapplikation som beräknar hur mycket material som behövs, bemöter denna lösning problemet på ett lämpligt sätt.

Slutsatser

När jag till en början lästa instruktionerna för denna projektuppgift kände jag att jag ville använda de kunskaper som jag har tagit till mig i kursen och lära mig något nytt. Därför valde jag att skapa en webbapplikation med ASP.NET och Razor Pages. Att jag valde detta tillvägagångssätt gjorde att jag nu har fått kunskaper i att utveckla webbapplikationer med programmeringsspråket C# och ramverket .NET, vilket jag känner att jag kommer ha en stor nytta av framöver.

Under projektet gick det mycket bra att utveckla webbapplikationen med ASP.NET och Razor Pages. Det var relativt enkelt att lära sig grunderna och det var flera aspekter som underlättade för mig som utvecklare. Ett exempel på detta är att jag fick grundstrukturen till alla mappar och filer. Däremot var det delvis utmanande eftersom jag aldrig har jobbat med detta tillvägagångssätt tidigare. Därmed var det mycket nytt att ta till sig och arbetet kan säkerligen bli ännu mer effektivt när jag har lärt mig mer.

En annan sak som jag anser var något problematisk är att jag inte hittade något effektivt sätt att live-koda. Varje gång jag hade gjort någon ändring i koden var jag tvungen att stänga ner webbapplikationen och sedan starta den igen för att ändringen skulle synas. Detta gjorde att arbetet vid framför allt stylingen av webbapplikationen blev väldigt tidsineffektiv. Därmed ska jag till nästa gång jag arbetar med ASP.NET och Razor Pages se om jag kan hitta något mer effektivt sätt för detta.

Något som jag gärna hade velat göra annorlunda i detta projekt var att webbapplikationen skulle vara lite bredare i sin funktionalitet. Nu kan användare räkna ut hur mycket material som går åt och hur många paket som ska köpas, men information om exempelvis hur mycket materialet kommer kosta saknas i webbapplikationen. En annan funktion som jag gärna skulle vilja lägga till är en till sida där användaren skulle kunna få inspiration i form av ett bildgalleri. Anledningen till att jag inte implementerat dessa funktioner är att min tid inte riktigt har tillåtit det. Men skulle jag gjort något annorlunda skulle jag velat implementera exempelvis kostnad och ett bildgalleri.

Källförteckning

[1] Byggstart. Renovera: En komplett guide (pris, regler, tips) [internet]. N/D.
https://www.byggstart.se/guide/renovera
Hämtad 2026-10-27.

[2] FLOQ. Allt om C# [internet]. N/D.
https://www.folq.se/c-sharp/
Hämtad 2026-10-27.

[3] IT-högskolan. Vad är .NET? [internet]. 2023-10-27.
https://www.iths.se/vad-ar-net/
Hämtad 2026-10-27.

[4] Limetta. Wireframe, designskiss och prototyp – vad är skillnaden? [internet]. N/D.
https://limetta.se/tips-metoder-for-digitala-projekt/wireframe-designskiss-och-prototyp----vad-ar-skillnaden/
Hämtad 2026-10-27.

[5] Hybrid Work. High fidelity prototype [internet]. N/D.
https://www.hybridwork.se/high-fidelity-prototype/
Hämtad 2026-10-27.

[6] Wikipedia. Flödesschema [internet]. 2025-05-18.
https://sv.wikipedia.org/wiki/Fl%C3%B6desschema
Hämtad 2026-10-27.

[7] Microsoft Learn. ASP.NET översikt [internet]. N/D.
https://learn.microsoft.com/sv-se/aspnet/overview
Hämtad 2026-10-27.

[8] Microsoft Learn. Razor-Pages arkitektur och begrepp i ASP.NET Core [internet]. N/D.
https://learn.microsoft.com/sv-se/aspnet/core/razor-pages/?view=aspnetcore-10.0&tabs=visual-studio
Hämtad 2026-10-27.

[9] Wikipedia. Objektorienterad programmering [internet]. 2025-04-17.
https://sv.wikipedia.org/wiki/Objektorienterad_programmering
Hämtad 2026-10-17.

[10] Wikipedia. Solid [internet]. 2022-12-13.
https://sv.wikipedia.org/wiki/SOLID
Hämtad 2026-10-27.

[11] Unga innovatörer. Brainstorming [internet]. N/D.
https://ungainnovatorer.se/brainstorming
Hämtad 2026-10-27.